

AI Accelerator Design - Assignment 3

BCHW $\leftrightarrow$ Bx(CHW) Tensor Flattening and Reconstruction

Roll No: CS26M205

1. Address mapping

Tensors are stored row-major (C order). For a fixed batch index b , element (c,h,w) lands in column j of the flattened row, where $j = c \cdot (H \cdot W) + h \cdot W + w$. The batch index is unchanged. The inverse recovers $c = j // (H \cdot W)$, $r = j \bmod (H \cdot W)$, $h = r // W$, $w = r \bmod W$. No reshape/view/flatten/ravel is used in the algorithm; numpy only allocates arrays and reads/writes single elements.

2. Pseudo code

```
FLATTEN(I[B,C,H,W]):
  F <- empty(B, C*H*W)
  for b in 0..B-1:
    for c in 0..C-1:
      for h in 0..H-1:
        for w in 0..W-1:
          j <- c*H*W + h*W + w
          F[b,j] <- I[b,c,h,w]
  return F

RECONSTRUCT(F[B,C*H*W], C,H,W):
  R <- empty(B,C,H,W)
  for b in 0..B-1:
    for j in 0..C*H*W-1:
      c <- j // (H*W)
      r <- j mod (H*W)
      h <- r // W
      w <- r mod W
      R[b,c,h,w] <- F[b,j]
  return R
```

3. Manual indexing example (B=1, C=2, H=2, W=3, row length 12)

(b,c,h,w)	$j = c*6 + h*3 + w$	j	value
(0,0,0,0)	$0*6 + 0*3 + 0$	0	1
(0,0,0,1)	$0*6 + 0*3 + 1$	1	2
(0,0,0,2)	$0*6 + 0*3 + 2$	2	3
(0,0,1,0)	$0*6 + 1*3 + 0$	3	4
(0,0,1,1)	$0*6 + 1*3 + 1$	4	5
(0,0,1,2)	$0*6 + 1*3 + 2$	5	6
(0,1,0,0)	$1*6 + 0*3 + 0$	6	7
(0,1,0,1)	$1*6 + 0*3 + 1$	7	8
(0,1,0,2)	$1*6 + 0*3 + 2$	8	9
(0,1,1,0)	$1*6 + 1*3 + 0$	9	10
(0,1,1,1)	$1*6 + 1*3 + 1$	10	11
(0,1,1,2)	$1*6 + 1*3 + 2$	11	12

Example: element (0,1,0,2) = 9 goes to $j = 1*6 + 0*3 + 2 = 8$ (the 9th slot, 0-indexed 8). Inverse: $8 // 6 = 1$ (c), $8 \bmod 6 = 2$, $2 // 3 = 0$ (h), $2 \bmod 3 = 2$ (w).

4. Experiments and results

Dataset/Input	B	C	H	W	E _{max}	MAE
MNIST / sample	1	1	28	28	0.0e+00	0.0e+00
CIFAR-10 / sample	1	3	32	32	0.0e+00	0.0e+00
Synthetic fmap1	2	1	16	16	0.0e+00	0.0e+00
Synthetic fmap2	2	3	16	16	0.0e+00	0.0e+00
Synthetic fmap3	2	8	16	16	0.0e+00	0.0e+00

Dataset/Input	B	C	H	W	E _{max}	MAE
Synthetic fmap4	2	16	16	16	0.0e+00	0.0e+00
Synthetic fmap5	2	32	16	16	0.0e+00	0.0e+00
Synthetic fmap6	2	64	16	16	0.0e+00	0.0e+00
Synthetic fmap7	2	128	16	16	0.0e+00	0.0e+00
Synthetic fmap8	2	256	16	16	0.0e+00	0.0e+00
Synthetic fmap9	2	500	16	16	0.0e+00	0.0e+00

MNIST/CIFAR rows use the real first training image when torchvision and a network are available; otherwise same-shaped random tensors are used (the row name says which). Synthetic feature maps use $B=2$, $H=W=16$ and $C \in \{1,3,8,16,32,64,128,256,500\}$. As an independent check (reference only, outside the algorithm) every flattened result was also compared with numpy reshape: all matched.

5. Discussion

Every case gives $E_{\max} = 0$ and $MAE = 0$, as expected: flatten/reconstruct only move values and never compute on them, and the map $j = c \cdot HW + h \cdot W + w$ is a bijection between $\{0..C-1\} \times \{0..H-1\} \times \{0..W-1\}$ and $\{0..CHW-1\}$. The result holds from $C=1$ to $C=500$. Run time grows linearly with the number of elements, $O(B \cdot C \cdot H \cdot W)$, as the loops visit each element exactly once. In hardware, the same mapping lets a systolic array be fed with contiguous addresses, and the multiply/add form can be implemented with a counter chain instead of a divider.